МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

КУРСОВАЯ РАБОТА

Программная модель управления оперативной памятью
тема работы

Руководитель _____ В.В. Тынченко
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151430 _____ В.Т. Гордов
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

Институт информатики и телекоммуникаций
Кафедра информатики и вычислительной техники

ЗАДАНИЕ

на курсовую работу по дисциплине «Алгоритмы и структуры данных»

студенту

Группа БПИ24-02 Форма обучения очная

1. Тема работы: Программная модель управления оперативной памятью

2. Срок сдачи студентом работы 30.12.25

3. Перечень вопросов, подлежащих разработке при написании теоретической части:

Анализ архитектурных особенностей оперативной памяти в современных вычислительных системах; Обзор подходов к программному управлению памятью; Исследование алгоритмов размещения блоков памяти; Обоснование выбора метода реализации программной модели.

4. Перечень вопросов, подлежащих разработке при написании практической части (либо указать номер варианта задания)

Описание общей характеристики программы и основных компонентов; Реализация пользовательского интерфейса; Проведения тестирования программы на различных тест-кейсах; Описание порядка эксплуатации программы; Проведение вычислительных экспериментов; Формирование выводов по результатам использования программы.

5. Дата выдачи задания: 13.09.25

Руководитель: Тынченко В.В. _____
Подпись

Задание принял к исполнению (дата): 13.09.25 _____
(подпись студента)

Содержание

Введение 5
1 Анализ предметной области 6
1.1 Роль и архитектурные особенности оперативной памяти в современных вычислительных системах 6
1.2 Основные подходы к программному управлению памятью 7
1.2.1 Управление на основе кучи (*heap-based allocation*). 7
1.2.2 Пул памяти (*memory pool*). 8
1.2.3 Сборка мусора (*garbage collection*). 8
1.3 Программные модели управления памятью: определение, назначение и примеры реализации 8
1.4 Метаданные и структуры данных в аллокаторах памяти 9
1.5 Обоснование выбора метода и средств реализации 10
1.6 Выводы 10
2 Описание программы 12
2.1 Общая характеристика 12
2.2 Структура программы и данных 12
2.3 Интерфейс программы 14
2.3.1 Пункт 1 — добавление блока памяти. 14
2.3.2 Пункт 2 — Освобождение блока памяти. 14
2.3.3 Пункт 3 — Вывод статистики кучи. 15
2.3.4 Пункт 4 — Выход из программы. 15
2.4 Тестирование программы 15
2.5 Выводы 16
3 Использование программы 17
3.1 Порядок эксплуатации программы 17
3.1.1 Инициализация кучи 17
3.1.2 Формирование сценария нагрузки 17
3.1.3 Мониторинг состояния кучи 17
3.1.4 Анализ результатов 17
3.2 Сценарии практического использования 18
3.2.1 Последовательное выделение и освобождение. 18
3.2.2 Имитация фрагментации. 18

3.2.3 Полное освобождение и повторное использование 18
3.3 Вычислительный эксперимент по оценке фрагментации..... 18
3.4 Выводы 20
Заключение 21
Список использованных источников 22
Приложение А 23
Листинг программы 23
Файл MemoryAllocator.h 23
Файл MemoryAllocator.cpp 25
Файл main.cpp 29
Приложение В — Проверка на оригинальность 32

Введение

Управление оперативной памятью — одна из фундаментальных задач в области системного и прикладного программирования. Эффективное распределение и освобождение памяти оказывают прямое влияние на затратность, производительность и работоспособность программы. Понимание принципа работы аллокаторов памяти и устройство оперативной памяти является особенно важным в условиях постоянного усложнения ПО.

Целью данной работы — создание и анализ программной модели управления оперативной памятью, которая реализует основные механизмы динамического распределения памяти на основе кучи.

Для достижения поставленной цели необходимо решить следующие задачи:

- анализ архитектурных особенностей оперативной памяти;
- изучение алгоритмов размещения блоков памяти;
- разработка модели аллокатора памяти на языке C++;
- вычислительный эксперимент по оценке фрагментации.

1 Анализ предметной области

1.1 Роль и архитектурные особенности оперативной памяти в современных вычислительных системах

Оперативная память (ОЗУ, *RAM — Random Access Memory*) представляет собой энергозависимое устройство хранения данных, предназначенное для временного размещения информации, активно используемой центральным процессором (ЦП) в процессе выполнения программ. В отличие от долговременных носителей (жёстких дисков, *SSD*), ОЗУ обеспечивает значительно более высокую скорость доступа — порядка десятков наносекунд на операцию чтения/записи, что делает её критически важной для производительности всей вычислительной системы.

В архитектуре фон Неймана, лежащей в основе подавляющего большинства современных компьютеров, ОЗУ служит единым адресным пространством как для исполняемого кода, так и для обрабатываемых данных. Это создаёт необходимость в строгом разделении и защите областей памяти, принадлежащих различным процессам, а также в эффективном управлении её физическими и логическими ресурсами.

Современные процессоры взаимодействуют с памятью через многоуровневую иерархию кэшей, что дополнительно усложняет задачу управления: локальность данных (пространственная и временная) напрямую влияет на производительность. Неправильно организованное распределение памяти может привести к частым промахам кэша, что нивелирует преимущества быстродействия ОЗУ.

Управление памятью осуществляется на трёх основных уровнях:

– **аппаратном**: через контроллер памяти и *Memory Management Unit* (*MMU*), отвечающую за трансляцию виртуальных адресов в физические;

– **системном**: операционной системой, реализующей механизмы виртуальной памяти, страничной адресации, защиты и изоляции процессов;

– **программном**: прикладными программами и библиотеками времени выполнения, реализующими аллокаторы кучи, пулы памяти и другие модели распределения.

В процессе управления памятью важную роль играют количественные оценки эффективности аллокатора. Одной из ключевых метрик является степень фрагментации.

Внешняя фрагментация возникает, когда свободная память распределена по множеству мелких блоков, не пригодных для выделения под крупный запрос, несмотря на достаточный общий объём свободной памяти. Степень внешней фрагментации можно оценить по формуле (1):

$$F_{внеш} = 1 - \frac{B\max}{M\mathrm{free}}, \quad (1)$$

где $B\max$ — размер наибольшего непрерывного свободного блока,

*M*free — общий объём свободной памяти.

Чем ближе значение *Fвнеш* к 1, тем выше фрагментация и тем менее эффективен аллокатор.

Для целей курсовой работы, посвящённой *программной модели*, основное внимание уделяется именно последнему уровню — алгоритмам и структурам данных, реализуемым в пользовательском пространстве для имитации или упрощённого воспроизведения процессов управления памятью. Такие модели позволяют исследовать фундаментальные проблемы, такие как фрагментация, накладные расходы и производительность, без необходимости взаимодействия с ядром операционной системы.

1.2 Основные подходы к программному управлению памятью

Программное управление памятью — это совокупность методов, алгоритмов и структур данных, реализуемых в программном коде для динамического выделения, использования и освобождения блоков памяти в рамках заранее выделенного буфера (например, массива байтов). Такой подход широко применяется как в учебных целях (для демонстрации принципов работы аллокаторов), так и в промышленных системах (например, в игровых движках, встраиваемых устройствах, высоконагруженных серверах).

Существует несколько ключевых подходов к организации программного управления памятью.

1.2.1 Управление на основе кучи (*heap-based allocation*).

Это наиболее универсальный и распространённый подход, используемый в языках с ручным управлением памятью, таких как C и C++. Память выделяется из непрерывной области — кучи. Аллокаторы кучи используют различные стратегии размещения блоков:

– ***first-fit*** — выделяет первый подходящий по размеру свободный блок;

– ***best-fit*** — ищет наименьший подходящий блок, минимизируя внешнюю фрагментацию;

– ***worst-fit*** — выделяет самый большой свободный блок (редко используется из-за высокой фрагментации);

– ***next-fit*** — модификация *first-fit*, начинающая поиск не с начала, а с последнего выделенного блока.

Эти стратегии различаются по производительности и степени фрагментации. В таблице 1 представлена их сравнительная характеристика.

Таблица 1 – сравнительная характеристика стратегий размещения блоков памяти

Стратегия	Сложность	Фрагментация	Производительность
First-fit	O(n)	Умеренная	Высокая
Best-fit	O(n)	Высокая (мелкие дыры)	Средняя
Worst-fit	O(n)	Очень высокая	Низкая
Next-fit	O(n)	Умеренная	Высокая

1.2.2 Пул памяти (*memory pool*).

Предварительно выделяется фиксированный объём памяти, разбитый на блоки одинакового размера. Такой подход исключает внутреннюю фрагментацию и обеспечивает O(1) сложность выделения/освобождения. Широко применяется в системах реального времени, сетевых стеках и при работе с объектами фиксированного размера (например, пакеты данных, узлы дерева).

1.2.3 Сборка мусора (*garbage collection*).

Автоматический подход, используемый в языках высокого уровня (*Java, Python, C#*). Сборщик мусора отслеживает достижимость объектов и освобождает недостижимые. Хотя этот метод устраняет необходимость ручного управления, он вносит непредсказуемую задержку *(«паузы»*) и требует дополнительных накладных расходов на отслеживание ссылок и копирование объектов.

1.2.4 Стековое управление.

Память выделяется и освобождается в порядке *LIFO* (*Last In — First Out*). Эффективно для локальных переменных и рекурсивных вызовов, но не подходит для динамических структур данных с произвольным временем жизни.

1.3 Программные модели управления памятью: определение, назначение и примеры реализации

Под *программной моделью управления оперативной памятью* в контексте данной работы понимается упрощённая имитация механизмов выделения и освобождения памяти, реализованная в виде автономной программы или библиотеки. Такая модель не взаимодействует напрямую с физической памятью, а оперирует выделенным заранее буфером (например, массивом байтов), внутри которого симулирует поведение реального аллокатора.

Основные цели создания программных моделей:

– **образовательные**: наглядная демонстрация принципов фрагментации, алгоритмов размещения, работы с метаданными;

– **исследовательские**: сравнение эффективности различных стратегий выделения памяти по критериям: скорость, фрагментация, накладные расходы;

– **практические**: разработка специализированных аллокаторов для встраиваемых систем, игровых движков, высоконагруженных серверов.

В реальных системах программные модели управления памятью реализованы в виде библиотечных аллокаторов. Например:

– ***dlmalloc* (*Doug Lea's malloc*)** — один из самых известных аллокаторов, лежащий в основе *malloc* в *glibc*. Использует двоичные деревья для больших блоков и связные списки для мелких;

– ***jemalloc*** — разработан для FreeBSD и используется в *Firefox* и *Redis*. Оптимизирован для многопоточности и минимизации фрагментации;

– ***tcmalloc* (*Thread-Caching Malloc*)** — разработан *Google*, использует кэширование памяти на уровне потоков для снижения конкуренции.

Эти реализации демонстрируют, что даже в *«стандартных»* библиотеках управление памятью — это сложная инженерная задача, требующая компромиссов между скоростью, памятью и масштабируемостью.

В учебной модели, как правило, упрощаются следующие аспекты:

– отсутствие поддержки многопоточности;
– фиксированный размер управляемого буфера;
– отсутствие взаимодействия с ОС;
– упрощённые структуры метаданных.

Тем не менее, даже такая модель позволяет продемонстрировать ключевые явления: внешнюю и внутреннюю фрагментацию, необходимость хранения заголовков блоков, важность стратегии слияния соседних свободных блоков.

1.4 Метаданные и структуры данных в аллокаторах памяти

Любой аллокатор памяти должен хранить информацию о состоянии каждого блока — занят он или свободен, каков его размер. Эта информация называется метаданными и обычно размещается непосредственно перед самим блоком данных (в «заголовке»).

Типичный заголовок может содержать:

– размер блока (в байтах);
– флаг занятости (часто кодируется в младшем бите размера, если выравнивание гарантирует чётность адреса);
– указатели на следующий/предыдущий свободный блок (в случае связного списка).

Для представления свободных блоков используются различные структуры данных:

– **односвязный** или двусвязный список — прост и эффективен для небольших объёмов памяти;
– **двоичное дерево поиска** — позволяет быстро находить блоки нужного размера (используется в *dlmalloc* для больших блоков);
– **хеш-таблицы по размеру** — применяются в некоторых специализированных аллокаторах.

1.5 Обоснование выбора метода и средств реализации

Для практической части курсовой работы целесообразно реализовать упрощённый аллокатор памяти на основе кучи, поддерживающий:

– динамическое выделение блоков произвольного размера;
– освобождение ранее выделенных блоков;
– автоматическое слияние соседних свободных блоков для борьбы с внешней фрагментацией;
– хранение метаданных непосредственно перед каждым блоком данных.

В качестве стратегии размещения выбран алгоритм *first-fit*, поскольку он:

– прост в реализации;
– обеспечивает приемлемую производительность (O(n) в худшем случае);
– позволяет наглядно продемонстрировать процессы фрагментации и дефрагментации.

Язык реализации — C++, так как он:

– предоставляет низкоуровневый контроль над памятью;
– широко используется для разработки системного ПО;
– позволяет использовать современные языковые средства (например, умные указатели).

Для представления свободных блоков будет использоваться односвязный список, что соответствует целям работы и позволяет применить базовые операции: обход, вставка, удаление.

1.6 Выводы

В ходе анализа предметной области установлено, что управление оперативной памятью — многоуровневая задача, охватывающая аппаратные, системные и программные аспекты. Для целей курсовой работы релевантен программный уровень, где реализуются алгоритмы динамического распределения памяти.

Рассмотрены основные подходы к программному управлению памятью: куча, пулы, сборка мусора, стек. Наиболее подходящим для учебно-исследовательской задачи признан аллокатор на основе кучи с использованием стратегии *first-fit* и представлением свободных блоков в виде связного списка.

Проанализирована роль метаданных и структур данных в работе аллокаторов. Подчёркнута важность таких механизмов, как слияние соседних блоков и выравнивание памяти. Приведены примеры реальных реализаций (*dlmalloc*, *jemalloc*), демонстрирующие сложность промышленных решений.

Выбранный подход позволяет:

– продемонстрировать работу фундаментальных структур данных;
– проанализировать явления фрагментации;
– реализовать функционально завершённую модель, соответствующую целям курсовой работы.

Данный выбор обоснован как с методической, так и с практической точки зрения и будет положен в основу второй части курсовой работы — описания разработанной программы.

2 Описание программы

2.1 Общая характеристика

Разработанная программа является упрощённой моделью аллокатора памяти на языке C++. **Её задача** — эмуляция основных механизмов управления памятью в рамках фиксированного буфера. Такая модель позволяет смоделировать процессы распределения и фрагментации памяти.

Модель оперирует буфером размером **64 КБ** (по умолчанию), который инициализируется как единый свободный блок памяти. Выделение и освобождение памяти выполняется внутри этого буфера. Пользователь взаимодействует с программой через консольное меню, выбирая действия: выделить блок, освободить блок по индексу, вывести статус кучи.

Использование языка **C++** обосновано возможностью **инкапсуляции** логики аллокатора в класс, что повышает читаемость и структурированность кода, при этом сохраняя низкоуровневый контроль над памятью через указатели и арифметику адресов.

Отличительная особенность модели в сравнении со стандартными аллокаторами (такими как *malloc*) — отсутствие использования специализированных контейнеров данных для отслеживания свободных блоков. Вместо этого при каждом запросе на выделение памяти выполняется **линейный проход по структуре** от начала к концу, что реализует классическую стратегию *first-fit* по адресу.

Минимальные системные требования к программной платформе перечислены в таблице 2.

Таблица 2 – системные требования к программе

Компонент системы	Требования
Процессор	x86/x64, 1 ГГц и выше
Оперативная память	64 МБ и более
Операционная система	Windows 10/11, Linux, macOS
Компилятор	GCC 9+, Clang 10+, MSVC 2019+
Стандарт C++	C++17 и выше
Свободное место на диске	1 МБ

2.2 Структура программы и данных

Исходный код программы с поясняющими комментариями дан в приложении А.

Программа состоит из трёх файлов:

– ***MemoryAllocator.h*** — заголовочный файл, содержащий объявление класса *MemoryAllocator* и структуру заголовка блока ***BlockHeader***;

– ***MemoryAllocator.cpp*** — реализация алгоритмов управления памятью;

– ***main.cpp*** — точка входа в программу, реализация пользовательского интерфейса.

Основная логика инкапсулирована в классе ***MemoryAllocator***. Он управляет эмулируемой кучей с помощью следующих методов:

– ***allocate(size_t)*** — выделение блока памяти по стратегии first-fit;

– ***deallocate(void*)*** — освобождение блока с последующим отметкой его как свободный;

– ***printStatus()*** — выводит статистику кучи: объёмы задействованной/свободной памяти, количество использованных блоков, степень фрагментации;

– ***getFragmentation()*** — возвращает показатель фрагментации;

– ***coalesceHeap()*** — слияние соседних свободных блоков;

– различные вспомогательные методы, требуемые для работы вышеуказанных.

Основная единица буфера — заголовок блока памяти (***BlockHeader***). Он имеет следующие параметры:

– ***size*** — объём в байтах (включая заголовок);

– ***isAllocated*** — определяет, занят ли блок или свободен.

При выделении блока памяти:

– запрашиваемый размер увеличивается на размер заголовка и выравнивается до 8 байт (вспомогательный метод ***alignUp***);

– выполняется проход по куче от начала к концу;

– остаток достаточно велик — блок делится на два;

– возвращается указатель на данные блока (идущие сразу после заголовка).

При освобождении:

– блок помечается как свободный (***isAllocated = false***);

– происходит слияние соседних свободных блоков — ***coalesceHeap()***.

Программа хранит указатели в векторе ***allocatedBlocks***, что обспечивает возможность выбирать блок для освобождения по индексу элемента, а не по адресу, что защищает от некорректного ввода и упрощает интерфейс.

Ввод данных пользователем в консоль валидируется: проверяются корректность числа, диапазон индекса, ненулевой размер и т.д.

2.3 Интерфейс программы

Взаимодействие с программой осуществляется через консольное меню. После запуска пользователю доступны следующие опции:

– пункт 1: добавление блока памяти;
– пункт 2: освобождение блока памяти;
– пункт 3: вывод статистика кучи;
– пункт 0: выход из программы.

```
===== RAM Management Model =====

============= MENU =============
1. Allocate memory block
2. Deallocate memory block
3. Display heap status
0. Exit
Choose an action:
```

Рисунок 2.1 — Интерфейс программы

2.3.1 Пункт 1 — добавление блока памяти.

Пользователь вводит размер добавляемого блока. При успешном вводе адрес записывается в ***allocatedBlocks*** и выводится в консоль.

```
============= MENU =============
1. Allocate memory block
2. Deallocate memory block
3. Display heap status
0. Exit
Choose an action: 1
Enter block size (bytes): 1000
Allocated at: 0xa624a8
```

Рисунок 2.2 — Добавление блока памяти

2.3.2 Пункт 2 — Освобождение блока памяти.

Программа выводит задействованные блоки с индексами (содержимое ***allocatedBlocks***). Пользователь вводит индекс, и соответствующий блок освобождается.

```
============= MENU =============
1. Allocate memory block
2. Deallocate memory block
3. Display heap status
0. Exit
Choose an action: 2
Allocated blocks:
0: 0xa624a8
1: 0xa628a8
2: 0xa63090
Enter index to deallocate: 1
Block deallocated successfully.
```

Рисунок 2.3 — Освобождение блока памяти

2.3.3 Пункт 3 — Вывод статистики кучи.

Программа выводит детальную статистику используемой кучи:

– размер кучи (64 КБ);
– занятый объём
– свободный объём;
– свободные блоки;
– задействованные блоки;
– процент фрагментации кучи.

```
============= MENU =============
1. Allocate memory block
2. Deallocate memory block
3. Display heap status
0. Exit
Choose an action: 3

=== Heap Status ===
Total size: 65536 bytes
Allocated: 4048 bytes (6.17676%)
Free: 61488 bytes (93.8232%)
Free blocks: 2
Allocated blocks: 2
External fragmentation: 3.2917%
```

Рисунок 2.4 — Вывод статистики кучи

2.3.4 Пункт 4 — Выход из программы.

Все задействованные данные удаляются, осуществляется выход из программы.

```
============= MENU =============
1. Allocate memory block
2. Deallocate memory block
3. Display heap status
0. Exit
Choose an action: 0
Exiting the program...
```

Рисунок 2.5 — Выход из программы

Программа обрабатывает следующие ошибки ввода:

– недопустимый размер;
– попытка освободить недопустимый индекс;
– нечисловой ввод.

2.4 Тестирование программы

Цель тестирования программы — проверка работоспособности разработанной модели и последующее исправление ошибок в случае обнаружения таковых.

Тестирование проводилось методом «чёрного ящика», при котором тестировщику недоступен просмотр исходного кода. Использованные тест-кейсы перечислены в таблице 3.

Таблица 3 – результаты тестирования программы

Сценарий	Ожидаемый результат	Фактический результат
Выделение 1024 байт	Адрес блока выдан, статус обновлён	Успешно
Выделение 70000 байт	Ошибка: переполнение памяти	Сообщение выведено
Освобождение по корректному индексу	Блок освобождён, статус обновлён	Успешно
Освобождение по некорректному индексу	Ошибка: некорректный индекс	Сообщение выведено
Ввод нечислового значения в меню	Повторный запрос	Успешно
Освобождение при пустом списке блоков	Сообщение: «нет блоков»	Сообщение выведено

2.5 Выводы

В данном разделе описана программная модель управления оперативной памятью, реализованная на C++. Ключевые рассмотренные моменты:

– общая характеристика программы;
– структура класса ***MemoryAllocator*** и принцип хранения блоков памяти;
– проход по куче как реализация стратегии *first-fit*;
– удобный консольный интерфейс с выбором блоков по индексу;
– результаты тестирования, подтвердившие работоспособность модели.

Программа успешно демонстрирует ключевые аспекты управления памятью: фрагментацию, выравнивание, эффективность стратегии размещения. Реализация соответствует заявленным целям и задачам курсовой работы.

3 Использование программы

3.1 Порядок эксплуатации программы

Программа предназначена для исследования поведения аллокатора памяти в реалистичных сценариях. Пользователь может использовать её как инструмент для анализа фрагментации, проверки устойчивости к ошибкам и оценки эффективности стратегии размещения *first-fit*.

Процесс работы с программой состоит из нескольких этапов.

3.1.1 Инициализация кучи

При запуске автоматически создаётся эмулируемая куча объёмом **65 536 байт** (64 КБ), представленная единым свободным блоком. Пользователь не взаимодействует с этим этапом — он происходит прозрачно.

3.1.2 Формирование сценария нагрузки

Пользователь самостоятельно определяет последовательность операций:

– выделение блоков с заданными размерами (например, чередование мелких и крупных запросов);

– освобождение блоков в произвольном или целенаправленном порядке (например, освобождение каждого второго блока для имитации «шахматной» фрагментации).

3.1.3 Мониторинг состояния кучи

В любой момент пользователь может вызвать команду **«Display heap status»**, чтобы получить количественные оценки:

– общий и свободный объём памяти;

– число занятых и свободных блоков;

– степень внешней фрагментации (в процентах).

3.1.4 Анализ результатов

На основе полученных данных пользователь может сделать следующие выводы:

– насколько эффективно используется память;

– возможно ли выделение крупного блока при наличии достаточного свободного объёма (признак фрагментации);

– как изменяется фрагментация при различных стратегиях освобождения.

Таким образом, программа позволяет проводить управляемые эксперименты, имитируя поведение реальных приложений, наглядно оценивая последствия принятых решений по управлению памятью.

3.2 Сценарии практического использования

Для демонстрации работы модели были подготовлены три сценария, реализуемых через консольный интерфейс. Каждый сценарий состоит из последовательности нескольких операций и демонстрирует определённый аспект модели памяти.

3.2.1 Последовательное выделение и освобождение.

– выделение блоков: 1024, 2048 и 512 байт;
– освобождение блока размером 2048 байт;
– выделение нового блока размером 1024 байт.

Результат: Программа успешно выделяет память. Новый добавляемый блок размером 1024 байт занимает место освобождённого блока (их адреса совпадают), что соответствует основной идее принципа ***first-fit*** — выделение первого подходящего свободного блока.

3.2.2 Имитация фрагментации.

– выделение 6 блоков по 10240 байт (61440 суммарно);
– освобождение блоков с чётными индексами (0, 2, 4);
– попытка выделить блок размером 20480 байт.

Результат: Несмотря на наличие свободной памяти после освобождения указанных блоков (35488 байт), выделение не удаётся, так как свободные блоки разделены занятыми и не образуют непрерывного участка. Степень фрагментации при этом составляет 71.78 %, что демонстрирует основную проблему явления внешней фрагментации.

3.2.3 Полное освобождение и повторное использование

– выделение произвольного количества блоков разного размера;
– освобождение всех блоков;
– вывод статистических данных кучи.

Результат: после полного освобождения кучи все блоки объединяются в один свободный объём. Степень внешней фрагментации составляет 0 %, что говорит о возвращении кучи в исходное состояние и демонстрирует работу механизма слияния блоков памяти.

3.3 Вычислительный эксперимент по оценке фрагментации

Для количественной оценки эффективности модели был проведён ручной вычислительный эксперимент, имитирующий типичное использование программы. Эксперимент состоял из **18** операций: ***allocate*** и ***deallocate*** блоков памяти с размерами от **256** до **8192** байт. При этом освобождение блоков происходило в **«шахматном» порядке**, в результате чего свободные блоки располагались между занятыми, что повышало степень фрагментации.

Эксперимент выполнялся через консольный интерфейс программы. После каждой операции фиксировались следующие данные:

– объём свободной памяти;
– объём занятой памяти;
– количество занятых блоков;
– количество свободных блоков;
– степень фрагментации.

Результаты эксперимента представлены в таблице 4.

Таблица 4 – результаты вычислительного эксперимента

Этап	Свободно, байт	Занято, байт	Занятых блоков	Свободных блоков	Фрагментация, %
1	63488	2048	1	1	0,00%
2	59392	6144	2	1	0,00%
3	58368	7168	3	1	0,00%
4	50176	15360	4	1	0,00%
5	49664	15872	5	1	0,00%
6	47616	17920	6	1	0,00%
7	43520	22016	7	1	0,00%
8	42496	23040	8	1	0,00%
9	44544	20992	7	2	4,60%
10	45568	19968	6	3	6,74%
11	46080	19456	5	4	7,78%
12	44032	21504	6	3	3,49%
13	48128	17408	5	3	11,70%
14	56320	9216	4	2	24,55%
15	58368	7168	3	2	27,19%
16	59392	6144	2	2	26,72%
17	61440	4096	1	2	29,17%
18	65536	0	0	1	0,00%

На рисунке 3.1 представлен график, отображающий динамику степени фрагментации по ходу выполнения эксперимента.

Анализ полученных результатов показывает, что:

– степень фрагментации увеличивается при каждом частичном освобождении блоков (чередование свободных с занятыми);
– степень фрагментации обнуляется при полном освобождении памяти, то есть при возврате модели в исходное состояние.

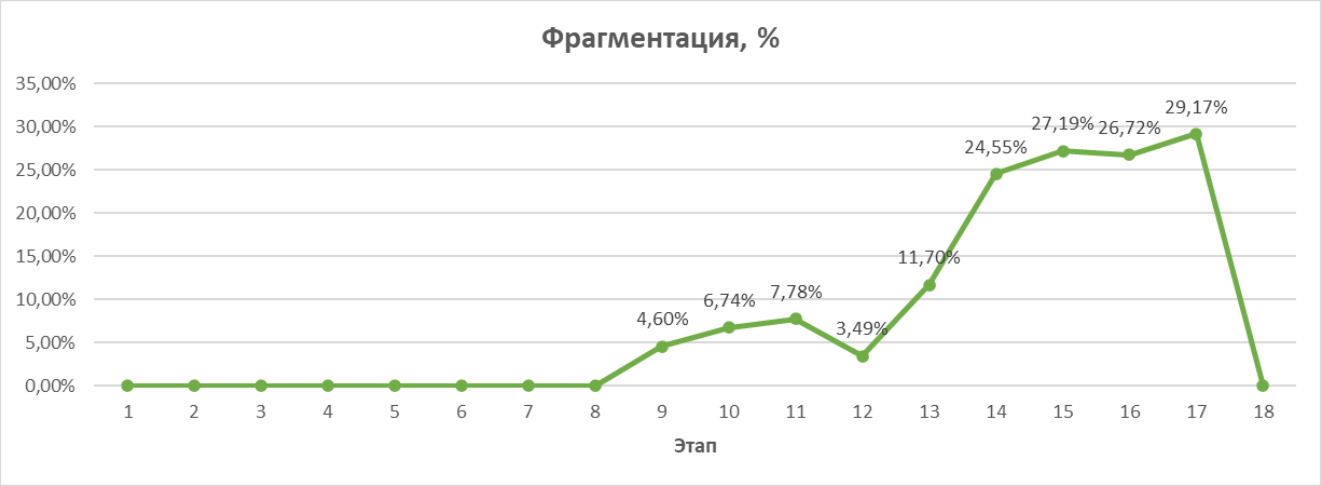


Рисунок 3.1 — Степень фрагментации

3.4 Выводы

Проведённый вычислительный эксперимент подтвердил корректность разработанной программной модели управления оперативной памятью. Реализованная стратегия ***first-fit*** в сочетании с механизмом автоматического слияния блоков обеспечивает:

– наглядную демонстрацию ключевых явлений использования памяти: внешняя фрагментация, слияния свободных блоков и нехватка памяти при наличии свободного объёма из-за фрагментации;

– полное устранение фрагментации при освобождении всех блоков за счёт автоматического слияния смежных участков;

– устойчивую работу при интенсивном чередовании операций выделения и освобождения, демонстрируя предсказуемое поведение и отсутствие сбоев.

Заключение

В ходе выполнения курсовой работы была разработана и исследована программная модель управления оперативной памятью, реализующая ключевые механизмы динамического распределения памяти на основе кучи. Модель корректно имитирует поведение реального аллокатора, включая выделение и освобождение блоков, хранение метаданных, выравнивание памяти и автоматическое слияние соседних свободных участков.

Достигнуты все поставленные во введении цели и задачи:

– проведён анализ архитектурных особенностей оперативной памяти и существующих подходов к её программному управлению (куча, пулы, сборка мусора);

– изучены и сравнены алгоритмы размещения блоков памяти, обоснован выбор стратегии *first-fit* для выполнения работы;

– разработан упрощённый аллокатор памяти на языке C++, инкапсулированный в класс ***MemoryAllocator***, с поддержкой односвязного списка свободных блоков и механизма полного перестроения (***rebuildFreeList***);

– проведены вычислительные эксперименты, наглядно продемонстрировавшие явление внешней фрагментации, его количественную оценку по формуле (1) и эффективное устранение за счёт слияния блоков.

Результаты тестирования и экспериментов подтвердили корректность и устойчивость модели: программа корректно обрабатывает граничные случаи (недостаток памяти, повторное освобождение, некорректный ввод), обеспечивает наглядный мониторинг состояния кучи и позволяет исследовать поведение аллокатора в различных сценариях нагрузки.

Возможные направления дальнейшего усовершенствования модели включают:

– добавление поддержки многопоточности с использованием примитивов синхронизации для оптимизации работы программы;

– реализация альтернативных стратегий размещения (*best-fit*, *buddy system*) для сравнительного анализа;

– интеграция графического интерфейса для визуализации состояния кучи в реальном времени.

Таким образом, курсовая работа представляет собой завершённое учебно-исследовательское решение, пригодное как для изучения основ управления памятью, так и для проведения экспериментов по оценке эффективности алгоритмов распределения памяти.

Список использованных источников

1. Филатов А. Ю., Михеев В. В. АНАЛИЗ ЭФФЕКТИВНОСТИ ПОТОКОВО-ЛОКАЛЬНОЙ СБОРКИ МУСОРА В РАСПРЕДЕЛЁННЫХ СИСТЕМАХ ХРАНЕНИЯ И ОБРАБОТКИ ДАННЫХ // Вестник СибГУТИ. 2022. №1 (57). URL: https://cyberleninka.ru/article/n/analiz-effektivnosti-potokovo-lokalnoy-sborki-musora-v-raspredelyonnyh-sistemah-hraneniya-i-obrabotki-dannyh (дата обращения: 04.12.2025).
2. Ивашенко В. П., Синцов С. В. Операции управления массивами данных в линейно адресуемой памяти // Доклады БГУИР. 2016. №6 (100). URL: https://cyberleninka.ru/article/n/operatsii-upravleniya-massivami-dannyh-v-lineyno-adresuemoy-pamyati (дата обращения: 04.12.2025).
3. Авдеева Яна Александровна, Прокин Александр Александрович Модель памяти в языках программирования // E-Scio. 2019. №1 (28). URL: https://cyberleninka.ru/article/n/model-pamyati-v-yazykah-programmirovaniya (дата обращения: 05.12.2025).
4. Дергачев А. М., Садырин Д. С., Ильина А. Г., Логинов И. П., Кореньков Ю. Д. Подход к верификации аллокаторов динамической памяти, основанный на символьном выполнении программ // Научно-технический вестник информационных технологий, механики и оптики. 2020. №1. URL: https://cyberleninka.ru/article/n/podhod-k-verifikatsii-allokatorov-dinamicheskoy-pamyati-osnovannyy-na-simvolnom-vypolnenii-programm (дата обращения: 18.12.2025).

Приложение А

Листинг программы

Приложение содержит исходные код разработанной программы с поясняющими комментариями на русском языке.

А.1 Файл MemoryAllocator.h

```
#ifndef MEMORY_ALLOCATOR_H
#define MEMORY_ALLOCATOR_H

#include <cstddef>
#include <iostream>

/**
 * @brief Структура заголовка блока памяти.
 *
 * Каждый блок (занятый или свободный) в эмулируемой куче начинается
 * с этого заголовка. Содержит информацию для взаимодействия с блоком.
 */
struct BlockHeader {
    size_t size; // Общий размер блока в байтах, включая сам заголовок
    bool isAllocated; // Флаг занятости: `true` — блок используется, `false` — сво-
боден
};

/**
 * @brief Класс программной модели управления оперативной памятью.
 *
 * Эмулирует работу аллокатора кучи на фиксированном буфере. Поддерживает:
 * - выделение памяти по стратегии first-fit,
 * - освобождение блоков,
 * - автоматическое слияние (coalescing) соседних свободных блоков,
 * - расчёт степени внешней фрагментации.
 */
class MemoryAllocator {
public:
    /**
     * @brief Конструктор аллокатора.
     * @param heapSize Размер эмулируемой кучи в байтах (по умолчанию 64 КБ = 65536
байт).
     */
    explicit MemoryAllocator(size_t heapSize = 65536);

    /**
     * @brief Деструктор.
     */
    ~MemoryAllocator();
```

```
    /**
     * @brief Выделяет блок памяти заданного размера.
     * @param userSize Требуемый размер данных в байтах (должен быть > 0).
     * @return Указатель на начало данных или `nullptr`, если память не может быть
выделена.
     */
    void* allocate(size_t userSize);

    /**
     * @brief Освобождает ранее выделенный блок памяти.
     * @param ptr Указатель, ранее полученный от метода `allocate()`.
     */
    void deallocate(void* ptr);

    /**
     * @brief Выводит детальную статистику о состоянии эмулируемой кучи.
     */
    void printStatus() const;

    /**
     * @brief Возвращает текущую степень внешней фрагментации.
     * @return Значение от 0.0 (полное отсутствие фрагментации)
     *         до 1.0 (максимальная фрагментация).
     */
    double getFragmentation() const;

    /**
     * @brief Преобразует указатель на данные в указатель на заголовок блока.
     * @param ptr Указатель, возвращённый `allocate()`, должен быть валидным.
     * @return Указатель на структуру `BlockHeader`, предшествующую данным.
     */
    BlockHeader* headerFromPtr(void* ptr) const;

    /**
     * @brief Проверяет, что указатель находится в пределах эмулируемой кучи.
     * @param ptr Проверяемый адрес.
     * @return `true` или `false` в зависимости от валидности адреса.
     */
    bool isValidPtr(void* ptr) const;

    /**
     * @brief Преобразует указатель на заголовок в указатель на данные.
     * @param header Указатель на заголовок блока.
     * @return Указатель на начало пользовательских данных (header + HEADER_SIZE).
     */
    void* ptrFromHeader(BlockHeader* header) const;

private:
    /// Выравнивание памяти по 8-байтной границе
```

```
    static constexpr size_t ALIGNMENT = 8;

    /// Размер заголовка блока (в байтах)
    static constexpr size_t HEADER_SIZE = sizeof(BlockHeader);

    /// Указатель на начало эмулируемой кучи (буфера в реальной памяти)
    char* heapStart;

    /// Общий размер эмулируемой кучи в байтах
    size_t heapSize;

    /**
     * @brief Округляет значение вверх до ближайшего кратного `alignment`.
     * @param value Исходное значение.
     * @param alignment Требуемое выравнивание (должно быть степенью двойки).
     * @return Выровненное значение.
     */
    size_t alignUp(size_t value, size_t alignment) const;

    /**
     * @brief Выполняет полное слияние соседних свободных блоков.
     */
    void coalesceHeap();
};

#endif
```

A.2 Файл MemoryAllocator.cpp

```
#include "MemoryAllocator.h"
#include <iostream>
using namespace std;

// === Конструктор и деструктор ===

MemoryAllocator::MemoryAllocator(size_t heapSize)
    : heapSize(heapSize) {
    heapStart = new char[heapSize];
    // Инициализируем один большой свободный блок
    BlockHeader* first = reinterpret_cast<BlockHeader*>(heapStart);
    first->size = heapSize;
    first->isAllocated = false;
}

MemoryAllocator::~MemoryAllocator() {
    delete[] heapStart;
}

// === Вспомогательные методы ===
```

```
size_t MemoryAllocator::alignUp(size_t value, size_t alignment) const {
    return (value + alignment - 1) & ~(alignment - 1);
}

void MemoryAllocator::coalesceHeap() {
    BlockHeader* curr = reinterpret_cast<BlockHeader*>(heapStart);
    while (curr < reinterpret_cast<BlockHeader*>(heapStart + heapSize)) {
        if (!curr->isAllocated) {
            // Ищем следующие смежные свободные блоки и объединяем
            BlockHeader* next = reinterpret_cast<BlockHeader*>(
                reinterpret_cast<char*>(curr) + curr->size
            );
            while (next < reinterpret_cast<BlockHeader*>(heapStart + heapSize) &&
                   !next->isAllocated) {
                curr->size += next->size;
                next = reinterpret_cast<BlockHeader*>(
                    reinterpret_cast<char*>(curr) + curr->size
                );
            }
        }
        curr = reinterpret_cast<BlockHeader*>(
            reinterpret_cast<char*>(curr) + curr->size
        );
    }
}

// === Основные методы аллокатора ===

void* MemoryAllocator::allocate(size_t userSize) {
    if (userSize == 0) return nullptr;

    size_t totalSize = alignUp(userSize + HEADER_SIZE, ALIGNMENT);
    BlockHeader* curr = reinterpret_cast<BlockHeader*>(heapStart);

    while (curr < reinterpret_cast<BlockHeader*>(heapStart + heapSize)) {
        if (!curr->isAllocated && curr->size >= totalSize) {
            size_t minSplitSize = alignUp(HEADER_SIZE, ALIGNMENT);
            if (curr->size >= totalSize + minSplitSize) {
                BlockHeader* newBlock = reinterpret_cast<BlockHeader*>(
                    reinterpret_cast<char*>(curr) + totalSize
                );
                newBlock->size = curr->size - totalSize;
                newBlock->isAllocated = false;

                curr->size = totalSize;
            }
            curr->isAllocated = true;
            return ptrFromHeader(curr);
        }
```

```
        curr = reinterpret_cast<BlockHeader*>(
            reinterpret_cast<char*>(curr) + curr->size
        );
    }

    cout << "Error: insufficient memory to allocate " << userSize << " bytes.\n";
    return nullptr;
}

void MemoryAllocator::deallocate(void* ptr) {
    if (!ptr || !isValidPtr(ptr)) {
        cout << "Error: invalid pointer for deallocation.\n";
        return;
    }

    BlockHeader* block = headerFromPtr(ptr);
    if (!block->isAllocated) {
        cout << "Error: block is already free.\n";
        return;
    }

    block->isAllocated = false;
}

// === Вспомогательные утилиты ===

bool MemoryAllocator::isValidPtr(void* ptr) const {
    char* cptr = static_cast<char*>(ptr);
    return (cptr >= heapStart + HEADER_SIZE) && (cptr < heapStart + heapSize);
}

BlockHeader* MemoryAllocator::headerFromPtr(void* ptr) const {
    return reinterpret_cast<BlockHeader*>(
        static_cast<char*>(ptr) - HEADER_SIZE
    );
}

void* MemoryAllocator::ptrFromHeader(BlockHeader* header) const {
    return static_cast<void*>(
        reinterpret_cast<char*>(header) + HEADER_SIZE
    );
}

// === Методы для отладки и анализа ===

void MemoryAllocator::printStatus() const {
    // Для корректного подсчёта фрагментации сначала сольём блоки
    MemoryAllocator* nonConstThis = const_cast<MemoryAllocator*>(this);
    nonConstThis->coalesceHeap();
```

```
    size_t totalFree = 0, totalAllocated = 0;
    size_t freeBlocks = 0, allocatedBlocks = 0;
    size_t maxFree = 0;

    BlockHeader* curr = reinterpret_cast<BlockHeader*>(heapStart);
    while (curr < reinterpret_cast<BlockHeader*>(heapStart + heapSize)) {
        if (curr->isAllocated) {
            totalAllocated += curr->size;
            allocatedBlocks++;
        } else {
            totalFree += curr->size;
            freeBlocks++;
            if (curr->size > maxFree) maxFree = curr->size;
        }
        curr = reinterpret_cast<BlockHeader*>(
            reinterpret_cast<char*>(curr) + curr->size
        );
    }

    double frag = (totalFree > 0) ? (1.0 - static_cast<double>(maxFree) / total-
Free) : 0.0;

    cout << "\n=== Heap Status ===\n";
    cout << "Total size: " << heapSize << " bytes\n";
    cout << "Allocated: " << totalAllocated << " bytes ("
         << (100.0 * totalAllocated / heapSize) << "%)\n";
    cout << "Free: " << totalFree << " bytes ("
         << (100.0 * totalFree / heapSize) << "%)\n";
    cout << "Free blocks: " << freeBlocks << "\n";
    cout << "Allocated blocks: " << allocatedBlocks << "\n";
    cout << "External fragmentation: " << (frag * 100) << "%\n";
}

double MemoryAllocator::getFragmentation() const {
    MemoryAllocator* nonConstThis = const_cast<MemoryAllocator*>(this);
    nonConstThis->coalesceHeap();

    size_t totalFree = 0, maxFree = 0;
    BlockHeader* curr = reinterpret_cast<BlockHeader*>(heapStart);
    while (curr < reinterpret_cast<BlockHeader*>(heapStart + heapSize)) {
        if (!curr->isAllocated) {
            totalFree += curr->size;
            if (curr->size > maxFree) maxFree = curr->size;
        }
        curr = reinterpret_cast<BlockHeader*>(
            reinterpret_cast<char*>(curr) + curr->size
        );
    }
    return (totalFree > 0) ? (1.0 - static_cast<double>(maxFree) / totalFree) :
0.0;
```

```
}
```

А.3 Файл main.cpp

```
#include "MemoryAllocator.cpp"
#include <iostream>
#include <vector>
#include <limits>

using namespace std;

/**
 * @brief Консольный интерфейс для тестирования аллокатора.
 *
 * Позволяет:
 *
 - выделять блоки заданного размера,
 *
 - освобождать их по индексу из списка,
 *
 - просматривать статистику кучи.
 *
 * Все выделенные указатели хранятся в векторе `allocatedBlocks` для удобства осво-
бождения.
 */
int main() {
    constexpr size_t HEAP_SIZE = 65536; // Размер эмулируемой кучи — 64 КБ
    MemoryAllocator allocator(HEAP_SIZE); // Инициализируем аллокатор
    vector<void*> allocatedBlocks; // Хранит указатели на все выделенные блоки

    int choice;
    size_t size;

    while (true) {
        cout << "\n===== RAM Management Model =====\n\n";
        cout << "============= MENU =============" << endl;
        cout << "1. Allocate memory block" << endl;
        cout << "2. Deallocate memory block" << endl;
        cout << "3. Display heap status" << endl;
        cout << "0. Exit" << endl;
        cout << "Choose an action: ";

        cin >> choice;

        // Обработка некорректного ввода (не число)
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

```
            cout << "ERROR: Invalid input. Please enter a number from the menu.\n";
            continue;
        }

        switch (choice) {
            case 1: { // Выделение блока памяти
                cout << "Enter block size (bytes): ";
                cin >> size;
                if (cin.fail() || size == 0) {
                    cin.clear();
                    cin.ignore(numeric_limits<streamsize>::max(), '\n');
                    cout << "ERROR: Invalid size.\n";
                    break;
                }
                void* ptr = allocator.allocate(size - 16);
                if (ptr) {
                    allocatedBlocks.push_back(ptr);
                    cout << "Allocated at: " << ptr << endl;
                }
                break;
            }

            case 2: { // Освобождение блока памяти по индексу
                if (allocatedBlocks.empty()) {
                    cout << "No allocated blocks available for deallocation.\n";
                    break;
                }
                cout << "Allocated blocks:" << endl;
                for (size_t i = 0; i < allocatedBlocks.size(); ++i) {
                    BlockHeader* block = allocator.headerFromPtr(allocated-
Blocks[i]);
                    cout << i << ": " << block << " (" << block->size << " bytes)"
<< endl;
                }
                cout << "Enter index to deallocate: ";
                int idx;
                cin >> idx;
                if (cin.fail() || idx < 0 || static_cast<size_t>(idx) >= allocated-
Blocks.size()) {
                    cin.clear();
                    cin.ignore(numeric_limits<streamsize>::max(), '\n');
                    cout << "ERROR: Invalid index.\n";
                    break;
                }
                allocator.deallocate(allocatedBlocks[idx]);
                allocatedBlocks.erase(allocatedBlocks.begin() + idx);
                cout << "Block deallocated successfully.\n";
                break;
            }
```

```
            case 3: { // Вывод статуса кучи
                allocator.printStatus();
                break;
            }

            case 0: { // Завершение программы
                cout << "Exiting the program...\n";
                return 0;
            }

            default: {
                cout << "ERROR: Invalid choice. Please select an option from the
menu.\n";
            }
        }
    }
}
```

Приложение В

Проверка на оригинальность

Проверка осуществлялась с помощью сервиса https://ap-vuz.ru.

Отчет о проверке на заимствования №1045957

Автор: Демонстрационный режим. Редактирование возможно в платном тарифе.
Проверяющий: Демонстрационный режим. Редактирование возможно в платном тарифе.

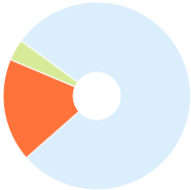
Организация: Демонстрационный режим. Редактирование возможно в платном тарифе.

ИНФОРМАЦИЯ О ДОКУМЕНТЕ

№ документа: 1045957
Начало загрузки: 2025-12-18 10:06:26
Длительность загрузки: 00:00:16
Имя исходного файла:
2025_12_18_05_06_24_69438bd0ce8ee.docx
Название работы:
2025_12_18_05_06_24_69438bd0ce8ee.docx
Размер текста: 142910 байт
Символов в тексте: 41489
Слов в тексте: 5064
Число предложений: 397

ИНФОРМАЦИЯ ОБ ОТЧЕТЕ

Последний готовый отчет (ред.)
Начало проверки: 2025-12-18 10:06:26
Длительность проверки: 00:01:06
Комментарии: не указано
Поиск с учетом редактирования: да
Модули поиска: ИПС Адилет, Библиография, Сводная коллекция ЭБС, Интернет Плюс, Сводная коллекция РГБ, Переводные заимствования (RuEn), Переводные заимствования по eLIBRARY.RU (EnRu), Переводные заимствования по Интернету (EnRu), Переводные заимствования издательства Wiley (RuEn), eLIBRARY.RU, СПС ГАРАНТ, Модуль поиска "КГАУ", Медицина, Диссертации НББ, Перефразирования по eLIBRARY.RU, Перефразирования по Интернету, Патенты СССР, РФ, СНГ, СМИ России и СНГ, Шаблонные фразы, Кольцо вузов, Издательство Wiley, Переводные заимствования, Коллекция НБУ, Перефразирования по коллекции издательства Wiley, Перефразирования по СПС ГАРАНТ: аналитика, СПС ГАРАНТ: аналитика, СПС ГАРАНТ: нормативно-правовая документация, Интернет



ЗАИМСТВОВАНИЯ	САМОЦИТИРОВАНИЯ	ЦИТИРОВАНИЯ	ОРИГИНАЛЬНОСТЬ	ИИ-КОНТЕНТ
17.8%	0%	3.67%	78.53%	0%